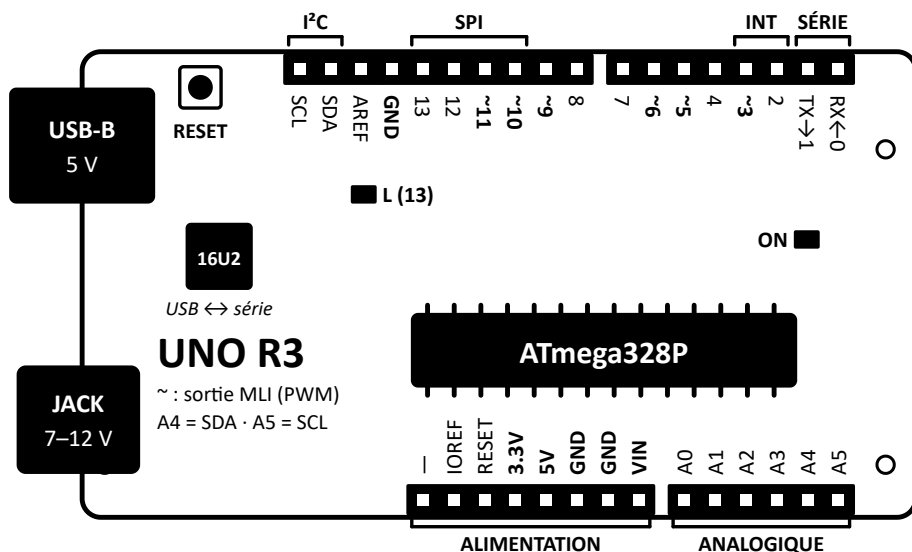


Arduino

Carte Uno · IDE 2 · électricité · entrées-sorties · langage · temps et mémoire · communication · capteurs et moteurs · dépannage

digitalWrite(13, HIGH) chasse fixe, gras : code à taper tel quel
b italique : valeur à remplacer (*b* : numéro de broche)
→ 127 valeur obtenue ; 1 renvoi aux explications
Outils > *Port* menu de l'IDE 2 (en français)
R4 particularité de l'Uno R4 · 1 piège ou danger

LA CARTE ARDUINO UNO R3



Vue de dessus, prise USB à gauche. Les broches SCL et SDA, près d'AREF, sont les mêmes signaux que A5 et A4. Les connecteurs de l'Uno R4 sont disposés de la même façon ; RESET redémarre le croquis, sans l'effacer.

QU'EST-CE QU'ARDUINO ?

- Une plateforme libre : des **cartes** à microcontrôleur, un **logiciel** (l'IDE Arduino) et un **langage** : du C++ enrichi de fonctions simples (**digitalWrite()**, **delay()**...).
- Le **microcontrôleur** exécute un seul programme, en boucle, sans système d'exploitation : il lit des **capteurs** (entrées) et commande des **actionneurs** (sorties).
- Le programme reste en mémoire flash : il redémarre seul à chaque mise sous tension, sans ordinateur.

L'UNO R3 EN CHIFFRES

Microcontrôleur	ATmega328P, 8 bits, 16 MHz
E/S numériques	14, dont 6 en MLI (~) · 5 V
Entrées analog.	6 (A0 à A5), 10 bits
Mémoires	flash 32 Ko (dont 0,5 Ko de chargeur) · SRAM 2 Ko · EEPROM 1 Ko
Alimentation	USB 5 V ou jack 7–12 V
Par broche	20 mA au plus

R4 Uno R4 (Minima, WiFi) : même format, même brochage ; Arm Cortex-M4 32 bits, 48 MHz, flash 256 Ko, RAM 32 Ko, USB-C, jack 6–24 V, **8 mA** par broche.

PREMIER CROQUIS : BLINK

```
// Blink : La LED clignote 1
const int LED = LED_BUILTIN; 2

void setup() { 3
  pinMode(LED, OUTPUT); 4
}

void loop() { 5
  digitalWrite(LED, HIGH); 6
  delay(1000); 7
  digitalWrite(LED, LOW);
  delay(1000);
}
```

- commentaire**, ignoré à la compilation ;
/* ... */ sur plusieurs lignes ;
- constante** nommée (**LED_BUILTIN** vaut 13 sur Uno) : la broche se change en un seul endroit ;
- setup()** s'exécute **une fois**, à la mise sous tension ou après RESET ;
- pinMode()** déclare la broche en **sortie** ;
- loop()** se répète **sans fin**, aussi vite que possible ;
- HIGH** = 5 V : LED allumée ; **LOW** = 0 V : éteinte ; chaque instruction finit par ; ;
- pause de 1 000 ms ; **delay()** **bloque** tout le programme (solution p. 5).

CE QUE L'IDE AJOUTE EN COULISSE

Un croquis est un dossier **Blink/** contenant **Blink.ino**. L'IDE ajoute **#include <Arduino.h>**, déclare les fonctions (appelables avant leur définition) et fournit **main()**, ici simplifié :

```
int main() {
  init(); // minuterias
  setup();
  for (;;) { // boucle sans fin
    loop();
    if (serialEventRun)
      serialEventRun(); // réception
  }
}
```

DU CODE À LA CARTE

- Brancher la carte avec un câble USB de **données** (un câble de charge seule ne fait apparaître aucun port).
- Choisir **carte** et **port** dans la liste de la barre d'outils, ou *Outils* > *Carte* puis *Outils* > *Port* (Windows : **COM3**, **COM4**...).
- Vérifier **Ctrl+R** : compile ; erreurs et taille s'affichent dans la zone *Sortie*.
- Téléverser **Ctrl+U** : compile puis envoie ; fin normale : « Téléversement fait. »
- Ouvrir le **Moniteur série** (**Ctrl+Shift+M**) au même débit que **Serial.begin()**.

Point de départ sûr :

Fichier > *Exemples* > *01.Basics* > *Blink*. Activer les avertissements : *Fichier* > *Préférences* > *Avertissements du compilateur* : **Tout** (Aucun par défaut).

RACCOURCIS DE L'IDE 2

Ctrl+R	Vérifier / Compiler
Ctrl+U	Téléverser
Ctrl+Shift+M	Moniteur série
Ctrl+Shift+I	Gérer les bibliothèques
Ctrl+T	Formatage automatique
Ctrl+/,	Commenter / décommenter
Ctrl+F Ctrl+G	Rechercher, occurrence suivante
Ctrl+L	Aller à la ligne n°...
Ctrl+Shift+C	Copier pour le forum (Markdown)
Ctrl+Alt+S	Exporter les binaires compilés
Ctrl+N Ctrl+S	Nouveau croquis, enregistrer

Sur Mac : **Cmd** au lieu de **Ctrl**.

RÈGLES D'OR !

- 20 mA** au plus par broche (Uno R3, Nano) ; **8 mA** sur Uno R4.
- Une LED a **toujours** sa résistance en série.
- Jamais de moteur, relais ou électroaimant directement sur une broche : transistor ou module (p. 2).
- Masse commune** : relier les GND de toutes les alimentations.
- Câbler carte **débranchée** ; vérifier avant de remettre sous tension.
- Carte 5 V (Uno, Nano, Mega) ≠ carte 3,3 V (Nano ESP32...) : jamais 5 V sur une broche 3,3 V.
- Broches 0 et 1 : liaison USB ; les laisser libres pour téléverser.
- Jamais de 230 V sur une plaque d'essai.

VOCABULAIRE

croquis	programme Arduino (<i>sketch</i>), fichier .ino
téléverser	envoyer le programme compilé dans la carte (<i>upload</i>)
chargeur d'amorçage	petit programme résident qui reçoit le croquis par USB (<i>bootloader</i>)
broche	borne de connexion (<i>pin</i>) : numérique 0–13, analogique A0–A5
MLI	modulation de largeur d'impulsion (<i>PWM</i>), p. 3
CAN	convertisseur analogique-numérique (<i>ADC</i>), p. 3
GND	masse : la référence 0 V
bouclier	carte d'extension enfichée sur les connecteurs (<i>shield</i>)
module	petit circuit prêt à câbler : capteur, écran, relais...
bibliothèque	code réutilisable, chargé par #include (p. 8)

2 · Électricité et composants

MÉMENTO — ARDUINO

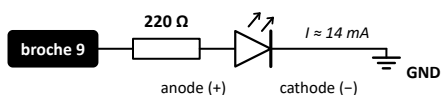
GRANDEURS ET LOIS

Tension U	V	volt : différence de « pression » entre deux points ; se mesure en parallèle
Intensité I	A	ampère : débit du courant ; se mesure en série
Résistance R	Ω	ohm : s'oppose au passage du courant
Puissance P	W	watt : énergie consommée par seconde
Capacité C	F	farad : réserve de charges d'un condensateur

Loi d'Ohm : $U = R \times I$; $P = U \times I$. Série : $R = R_1 + R_2$; parallèle : $1/R = 1/R_1 + 1/R_2$. Ex. : 5 V aux bornes de 220 $\Omega \rightarrow I = 5 / 220 \approx 22,7$ mA.

Le courant ne circule que dans une **boucle fermée**, du + vers le -, à travers la charge. Une boucle sans résistance est un **court-circuit** !

LED : LA RÉSISTANCE SÉRIE



$R = (U_{\text{alim}} - U_{\text{LED}}) / I$. Ex. : 5 V, LED rouge 2 V, 15 mA $\rightarrow (5 - 2) / 0,015 = 200 \Omega \rightarrow$ prendre la valeur normalisée **supérieure**, 220 $\Omega \rightarrow I \approx 13,6$ mA.

Rouge	$\approx 1,8-2,2$ V
Jaune, verte classique	$\approx 2,0-2,2$ V
Bleue, blanche, verte vive	$\approx 2,8-3,4$ V

Valeurs typiques : lire la fiche du composant. **Anode** (+) : patte longue ; **cathode** (-) : patte courte, côté méplat. Témoin lumineux : 220 Ω à 1 k Ω .

CODE DES COULEURS



1 2 chiffres · 3 multiplicateur · 4 tolérance (souvent or, $\pm 5\%$)

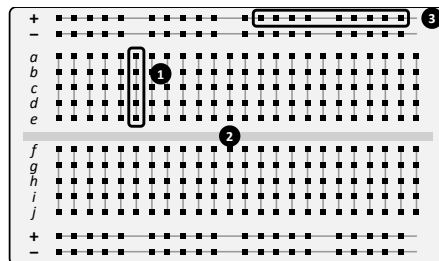
COULEUR	0-9	x	$\pm$
noir	0	$\times 1$	
marron	1	$\times 10$	$\pm 1\%$
rouge	2	$\times 100$	$\pm 2\%$
orange	3	$\times 1\text{ k}$	
jaune	4	$\times 10\text{ k}$	
vert	5	$\times 100\text{ k}$	$\pm 0,5\%$
bleu	6	$\times 1\text{ M}$	$\pm 0,25\%$
violet	7	$\times 10\text{ M}$	$\pm 0,1\%$
gris	8		$\pm 0,05\%$
blanc	9		
or		$\times 0,1$	$\pm 5\%$
argent		$\times 0,01$	$\pm 10\%$

Ne Mangez Rien Ou Je Vous Brûle Votre Grande Barbe : noir $\rightarrow$ blanc. Ex. : rouge-rouge-marron = 220 Ω ; marron-noir-rouge = 1 k Ω ; jaune-violet-rouge = 4,7 k Ω ; marron-noir-orange = 10 k Ω . Série E12 : 10 12 15 18 22 27 33 39 47 56 68 82.

MULTIMÈTRE

- Tension (V=) : pointes **en parallèle** sur le composant.
- Courant (A) : multimètre **en série**, pointe rouge dans la borne A ou mA !
- Résistance, continuité : circuit **hors tension**.

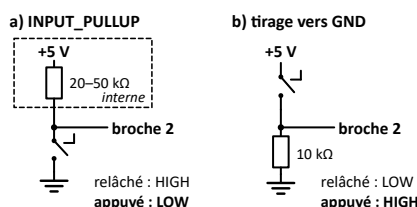
PLAQUE D'ESSAI



- les 5 trous d'une demi-colonne (a-e, f-j) sont reliés ;
- rien ne traverse la rainure : on y place les circuits intégrés et les boutons, à cheval ;
- les lignes + et - courent sur la longueur, parfois coupées au milieu des grandes plaques.

Convention : fil rouge pour le +, noir (ou bleu) pour la masse.

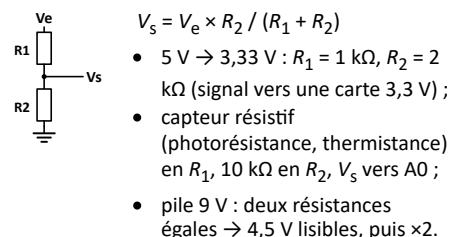
BOUTON POUSSOIR ET RÉSISTANCE DE TIRAGE



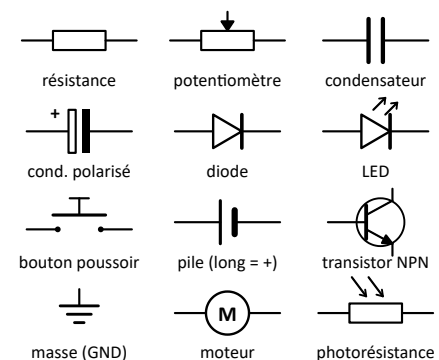
Une entrée reliée à rien **flotte** : elle lit n'importe quoi. On la **tire** vers un niveau connu. a) résistance interne (20 à 50 k Ω) activée par **INPUT_PULLUP** : logique **inversée** ; b) résistance externe de 10 k Ω vers la masse.

Bouton à 4 pattes : elles sont reliées deux à deux ; câbler deux pattes **en diagonale** pour être sûr.

DIVISEUR DE TENSION

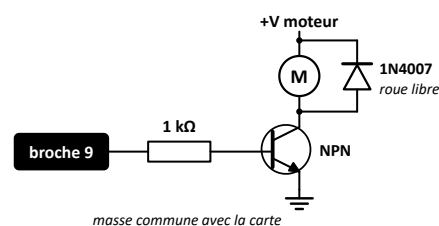


SYMBOLES DES SCHÉMAS



Symboles de la norme CEI, en usage en Europe ; aux États-Unis, la résistance est dessinée en zigzag.

COMMANDER UNE CHARGE



- Le **transistor** est un interrupteur commandé : quelques mA dans la base laissent passer le courant du moteur.
- Diode de roue libre**, en inverse aux bornes d'un moteur, d'un relais ou d'un électroaimant : elle absorbe la surtension à la coupure.
- Plus de courant : MOSFET *logic level* (commandable en 5 V, ex. IRLZ44N) ou module tout fait (pont en H, relais, p. 7).
- Alimentation séparée pour la charge, **masses reliées**.

ALIMENTER LA CARTE (UNO R3)

USB	5 V de l'ordinateur, en général 500 mA au plus
Jack	7 à 12 V, régulés à 5 V par la carte (Uno R4 : 6 à 24 V)
VIN	même plage que le jack ; aucune protection contre l'inversion de polarité
Broche 5V	sortie ; jusqu'à 1 A selon la source (doc. Arduino) ; ne pas y injecter de tension
Broche 3.3V	sortie de 150 mA au plus

Faire le **bilan** des courants ; moteurs, servos et rubans de LED : alimentation à part.

COMPOSANTS DE BASE

- Potentiomètre** : résistance réglable à 3 bornes ; le curseur, au milieu, forme un diviseur (10 k Ω courant).
- LED RVB** : 4 pattes ; cathode commune (vers GND) ou anode commune (vers 5 V, logique inversée).
- Condensateur céramique** 100 nF, près des circuits intégrés : découplage ; sans polarité.
- Condensateur chimique** : réserve d'énergie ; polarisé (bande -), peut éclater s'il est inversé.
- Diode 1N4007** : passage dans un seul sens ; la bague marque la cathode.
- Photorésistance, thermistance CTN** : leur résistance baisse avec la lumière, avec la chaleur.
- Buzzer** actif : son fixe, `digitalWrite()` ; passif : `tone()`.

UNITÉS ET PRÉFIXES

M méga	$\times 1\,000\,000$	1 M Ω = 1 000 k Ω
k kilo	$\times 1\,000$	4,7 k Ω = 4 700 Ω , noté 4k7
m milli	$\div 1\,000$	20 mA = 0,02 A
μ micro	$\div 1\,000\,000$	100 μ F ; 10 μ s
n nano	$\div 10^9$	100 nF = 0,1 μ F
p pico	$\div 10^{12}$	22 pF

Condensateur marqué **104** : 10 suivi de 4 zéros, en pF $\rightarrow$ 100 000 pF = 100 nF.

3 · Entrées et sorties

MÉMENTO — ARDUINO

ENTRÉES ET SORTIES NUMÉRIQUES

pinMode(b, OUTPUT) sortie : HIGH (5 V) ou LOW (0 V)

pinMode(b, INPUT) entrée sans tirage (état par défaut)

pinMode(b, INPUT_PULLUP) entrée tirée vers 5 V par une résistance interne

digitalWrite(b, HIGH) mettre la sortie à 5 V

digitalRead(b) lire l'entrée : HIGH ou LOW

digitalWrite(b, !digitalRead(b)) inverser l'état d'une sortie

- Seuils (ATmega328P sous 5 V) : LOW sous $\approx 1,5$ V, HIGH au-dessus de ≈ 3 V ; entre les deux : indéterminé.
- A0 à A5 servent aussi de broches numériques (= 14 à 19).
- Broche 13 : la LED intégrée fausse les lectures avec INPUT_PULLUP ❶

LIRE UN BOUTON (UN APPUI = UNE BASCULE)

```
const int BOUTON = 2; // vers GND
const int LED = 13;
bool allumee = false;
int avant = HIGH; // tour précédent

void setup() {
  pinMode(BOUTON, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
}

void loop() {
  int etat = digitalRead(BOUTON);
  if (etat == LOW && avant == HIGH) { ❶
    allumee = !allumee;
    digitalWrite(LED, allumee);
    delay(20); ❷
  }
  avant = etat;
}
```

- ❶ on réagit au **changement** (front descendant = appui), pas à l'état ;
- ❷ anti-rebond sommaire : un contact « rebondit » quelques ms ; version sans delay() p. 5.

ENTRÉES ANALOGIQUES

analogRead(A0) 0 à 1023 (CAN 10 bits) : 0 V $\rightarrow$ 0, 5 V $\rightarrow$ 1023 ; $\approx 4,9$ mV par pas ; $\approx 100 \mu\text{s}$ par lecture

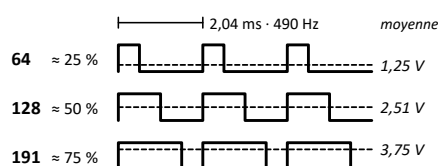
lecture * 5.0 / 1023.0 tension en volts (1023 dans les exemples Arduino, 1024 selon la fiche technique : écart < 0,1 %)

analogReference(INTERNAL) pleine échelle = 1,1 V (Uno R3) : plus fin pour les petites tensions

analogReference(EXTERNAL) pleine échelle = tension sur AREF (0 à 5 V)

- Jamais plus de 5 V, ni de tension négative, sur une entrée ❶
- Entrée non câblée : la valeur fluctue ; lecture bruitée : moyenne de plusieurs mesures.
- [R4] **analogReadResolution(14)** : jusqu'à 14 bits (10 par défaut) ; référence interne AR_INTERNAL = 1,5 V.

MLI (PWM) : SORTIE « PSEUDO-ANALOGIQUE »



analogWrite(9, v)
v de 0 (toujours 0 V) à 255 (toujours 5 V) ; pas besoin de pinMode

- Signal carré : la part du temps à 5 V (rapport cyclique) vaut $v/255$; une LED ou un moteur ne « voient » que la moyenne.
- Broches ~3, 5, 6, 9, 10, 11 (Uno R3 et R4) ; 490 Hz, 980 Hz sur 5 et 6 (Uno R3).
- Sur 5 et 6, la valeur 0 n'éteint pas toujours tout à fait (minuterie partagée avec millis()).
- [R4] vraie sortie analogique (CNA 12 bits) sur A0 avec **analogWrite()**.

CHANGER D'ÉCHELLE

```
int v = analogRead(A0); // 0...1023
int lum = map(v, 0, 1023, 0, 255);
analogWrite(9, lum); // 0...255
int x = map(512, 0, 1023, 0, 255);
// → 127
int inv = map(v, 0, 1023, 255, 0);
int c = constrain(lum + 50, 0, 255);
```

map() calcule en entiers (partie décimale tronquée) et ne borne pas : l'associé à **constrain()**, qui ramène la valeur entre deux bornes. Bornes d'arrivée inversées (255, 0) : sens inversé.

AUTRES FONCTIONS D'ENTRÉE-SORTIE

tone(b, 440) note (440 Hz) jusqu'à **noTone(b)**

tone(b, 440, 200) pendant 200 ms, sans bloquer le programme

pulseIn(b, HIGH) durée d'une impulsion en μs ; 0 si rien en 1 s

pulseIn(b, HIGH, 30000) délai d'attente réduit à 30 ms

random(1, 7) entier de 1 à 6 : la borne haute est exclue

randomSeed(analogRead(A5)) graine variable : broche non câblée

tone() : une seule note à la fois, 31 Hz au minimum ; perturbe la MLI des broches 3 et 11 (sauf Mega).

8 SORTIES AVEC 3 BROCHES : 74HC595

```
const int DATA = 11, CLK = 12, LAT = 8;
void afficher(byte motif) { // Q7...Q0
  digitalWrite(LAT, LOW);
  shiftOut(DATA, CLK, MSBFIRST, motif);
  digitalWrite(LAT, HIGH); // sorties
}
void setup() {
  pinMode(DATA, OUTPUT);
  pinMode(CLK, OUTPUT);
  pinMode(LAT, OUTPUT);
  afficher(0b10100101); // 4 sur 8
}
```

Un bit par sortie ; les 8 changent ensemble quand le verrou (LAT) remonte. Câblage : DS (14), SHCP (11), STCP (12) ; OE (13) à GND, MR (10) à 5 V.

INTERRUPTIONS EXTERNES

```
const byte CAPTEUR = 2; // 2 ou 3
volatile unsigned long compte = 0; ❶

void compter() { ❷
  compte++;
}

void setup() {
  Serial.begin(9600);
  pinMode(CAPTEUR, INPUT_PULLUP);
  attachInterrupt(
    digitalPinToInterrupt(CAPTEUR),
    compter, FALLING);
}

void loop() {
  noInterrupts(); ❸
  unsigned long copie = compte;
  interrupts();
  Serial.println(copie);
  delay(1000);
}
```

❶ **volatile** : variable partagée avec l'interruption ;

❷ routine **courte**, sans paramètre ni valeur renvoyée ; **delay()** n'y marche pas, **millis()** n'y avance pas ; éviter **Serial** ;

❸ une variable de plus d'un octet se lit en plusieurs instructions : copie protégée.

Déclencheurs : **LOW**, **CHANGE**, **RISE** (montée), **FALLING** (descente). Broches : Uno R3 et R4 : 2, 3 ; Mega : 2, 3, 18 à 21 ; Nano Every, Nano ESP32 : toutes.

QUI PARTAGE QUOI (UNO R3)

millis()	minuterie 0 ; MLI des broches 5 et 6
delay()	broches 5 et 6
bibliothèque Servo	minuterie 1 : supprime la MLI de 9 et 10, même sans servo dessus
tone()	minuterie 2 : MLI des broches 3 et 11
Serial	broches 0 et 1
SPI, Wire (I²C)	10 à 13 ; A4 et A5

MONTAGE COMPLET : VEILLEUSE AUTOMATIQUE

```
// Photorésistance : 5 V → A0
// 10 kΩ : A0 → GND ; LED + 220 Ω : 9
const int CAPTEUR = A0, LED = 9;
const int NUIT = 300, JOUR = 400; ❶
bool allumee = false;

void setup() {
  pinMode(LED, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int lum = analogRead(CAPTEUR);
  Serial.println(lum);
  if (lum < NUIT) allumee = true; ❷
  if (lum > JOUR) allumee = false; ❸
  digitalWrite(LED, allumee);
  delay(200);
}
```

❶ deux seuils : entre 300 et 400, rien ne change (**hystérésis**) ; la LED ne papillote pas au crépuscule ;

❷ lire les valeurs réelles au moniteur ou au traceur série pour régler les seuils ;

❸ plus de lumière $\rightarrow$ photorésistance plus faible $\rightarrow$ tension plus haute sur A0 $\rightarrow$ lecture plus grande.

4 · Le langage : types, opérateurs, structures

MÉMENTO — ARDUINO

TYPES DE DONNÉES (UNO R3, AVR 8 BITS)

TYPE	O.	VALEURS
bool	1	true / false
char	1	-128 à 127 ; un caractère : 'A'
byte, uint8_t	1	0 à 255
int, int16_t	2	-32 768 à 32 767 ❶
unsigned int	2	0 à 65 535
long, int32_t	4	-2 147 483 648 à 2 147 483 647
unsigned long	4	0 à 4 294 967 295 : millis()
float	4	$\pm 3,4 \times 10^{38}$, 6 à 7 chiffres significatifs
double	4	identique à float sur AVR ❶

Cartes 32 bits (Uno R4, ESP32...) : **int** = 4 octets, **double** = 8 octets. Pour un code portable : **uint8_t**, **int16_t**, **uint32_t**... (taille garantie).

CONSTANTES ET LITTÉRAUX

HIGH LOW true false	niveaux 1 et 0 ; valeurs booléennes
INPUT OUTPUT INPUT_PULLUP	modes de broche ; LED_BUILTIN : 13 sur Uno
0xFF 0b11111111	255 en hexadécimal, en binaire
60000UL 100000L	suffixes : unsigned long, long
3.0 1.5e3	nombres à virgule (float)
'A' "Uno"	un caractère ; une chaîne de 4 octets (zéro final)
const int LED = 9;	constante typée : à préférer
#define LED 9	remplacement de texte, sans type ni ;

STRUCTURES ET ÉNUMÉRATIONS

```
struct Mesure { // type composé
    float temp;
    int humidite;
};
Mesure m = {21.5, 40};

enum Mode { ARRET, AUTO, MANUEL };
Mode mode = AUTO; // 0, 1, 2

void setup() {
    m.temp = 22.0; // champ : Le point
    if (mode == AUTO) m.humidite = 45;
}
```

struct regroupe des valeurs liées ; **enum** donne des noms à des états (voir la machine à états, p. 5).

BITS ET OCTETS

bitRead(x, n)	bit n de x : bitRead(10, 1) → 1
bitSet bitClear bitWrite	mettre un bit à 1, à 0, à une valeur
bit(3)	$1 \ll 3 \rightarrow 8$
lowByte highByte	octet bas, haut : 0x1234 → 0x34, 0x12
x & 0x0F	masque : garder les 4 bits bas
x = (1 << 3)	forcer le bit 3 à 1
x &= ~(1 << 3)	forcer le bit 3 à 0

OPÉRATEURS

+ - * / %	arithmétique ; 7 / 2 → 3 (entiers) ; 7 % 2 → 1 ; 7 / 2.0 → 3.5
== != < > <= >=	comparaison
&& !	et, ou, non (logique)
& ^ ~ << >>	et, ou, ou exclusif, inversion, décalages (bit à bit)
= += -= *= /=	affectation, affectations combinées
++ --	ajouter, retirer 1
c ? a : b	vaut a si c est vrai, sinon b

STRUCTURES DE CONTRÔLE

```
if (t > 30) {
    ventiler();
} else if (t < 18) {
    chauffer();
} else {
    arreter();
}

switch (mode) {
    case 1: clignoter(); break;
    case 2: fixe(); break;
    default: eteindre();
}

for (int i = 0; i < 10; i++) { // 0...9
    Serial.println(i);
}
while (digitalRead(2) == HIGH) {}
```

- switch** : sans **break**, l'exécution continue dans le cas suivant ❶
- while** vide : attendre l'appui (bloquant) ; **do { ... } while (...)** ; fait au moins un tour.
- break** sort d'une boucle ; **continue** passe au tour suivant ; **return** quitte la fonction.

FONCTIONS

```
float celsius(float f) {
    return (f - 32) * 5 / 9;
}

void clignoter(int b, int fois) {
    for (int i = 0; i < fois; i++) {
        digitalWrite(b, HIGH); delay(100);
        digitalWrite(b, LOW); delay(100);
    }
}

void echanger(int &a, int &b) {
    int t = a; a = b; b = t;
}
```

- return** renvoie la valeur : **celsius(212) → 100** ; **void** : ne renvoie rien.
- Les paramètres sont des **copies** ; avec **&** (référence), **echanger(x, y)** modifie bien x et y.

PORTÉE DES VARIABLES

```
int total = 0; // globale

void loop() {
    int n = 0; // locale
    static int k = 0; // statique
    n++; // vaut toujours 1
    k++; // 1, 2, 3...
```

Globale : visible partout, existe tout le temps.
Locale : visible dans son bloc { }, recrée à chaque passage. **Statique** : locale qui garde sa valeur d'un appel à l'autre. Locale non initialisée : valeur **imprévisible** ❶

TABLEAUX ET CHÂÎNES

```
int broches[] = {3, 5, 6, 9};
const int N = sizeof(broches) / sizeof(broches[0]); // 4
for (int i = 0; i < N; i++) {
    pinMode(broches[i], OUTPUT);
}

char nom[8] = "Uno"; // + '\0' final
char txt[20];
sprintf(txt, sizeof(txt), "A0=%d", analogRead(A0));
dtostrf(21.537, 5, 1, txt); → " 21.5"

String msg = "T = "; // pratique...
msg += 21.5; // ...mais gourmand
```

- Indices de 0 à N - 1 ; **aucun contrôle** : **broches[4]** écrase la mémoire ❶
- sprintf()** ne connaît pas %f sur AVR : convertir d'abord avec **dtostrf()** (valeur, largeur, décimales, tampon).
- String** : **length()**, **toInt()**, **toFloat()**, **indexOf()**, **substring()**, **trim()**... À éviter dans un programme qui tourne longtemps sur 2 Ko de SRAM.

FONCTIONS MATHÉMATIQUES ET CONVERSIONS

abs(-5) min(a, b) max(a, b)	valeur absolue → 5 ; plus petit, plus grand
pow(2, 10) sq(3) sqrt(2)	puissance → 1024.0 ; carré → 9 ; racine carrée
round(2.5)	arrondi → 3
sin(PI / 2)	trigonométrie en radians → 1.0 ; DEG_TO_RAD convertit les degrés
(int)3.9	→ 3 : la conversion en entier tronque
(float)7 / 2	→ 3.5 : convertir avant de diviser
atoi("42")	texte → entier : 42
String(42)	nombre → texte "42"

Sur AVR, **abs**, **min**, **max**, **sq** sont des macros : ne pas y écrire **i++** ou un appel de fonction, évalués deux fois ❶

PIÈGES DU LANGUAGE !

```
int x = 1000 * 60;
→ -5536 : le calcul déborde d'un int 16 bits ; écrire 1000UL * 60

unsigned long t = 60 * 1000;
→ 4294961760 : le calcul se fait en int avant la conversion
1 / 3 * 3.0
→ 0 : 1 / 3 est une division entière

if (x = 5)
    affectation, toujours vraie : comparer avec ==

if (x > 3);
    le ; termine le if : le bloc suivant s'exécute toujours

a == b (float)
9.0 / 0.3 ne vaut pas exactement 30 :
tester fabs(a - b) < 0.001

int t = millis();
faux au bout de 32,7 s : toujours
unsigned long
digitalwrite(...)
la casse compte : digitalWrite
```

Messages du compilateur correspondants : p. 8 ; avertissements à activer (p. 1).

5 · Temps, programmes réactifs, mémoire

MÉMENTO — ARDUINO

MESURER LE TEMPS

delay(ms) pause **bloquante** : plus rien ne se passe, sauf les interruptions

delayMicroseconds(μs) pause courte, précise de 3 à 16 383 μs

millis() ms écoulées depuis le démarrage (**unsigned long**) ; repasse par 0 après ≈ 49,7 jours

micros() μs ; par pas de 4 μs (16 MHz) ; repasse par 0 après ≈ 71,6 min

DURÉES EN MILLISECONDES

1000 1 s (= 1 000 000 μs)

60UL * 1000 1 min = 60 000 ms

60UL * 60 * 1000 1 h = 3 600 000 ms

24UL * 60 * 60 * 1000 1 jour = 86 400 000 ms

Le suffixe UL sur le **premier** nombre fait tout le calcul en **unsigned long** (sinon débordement, p. 4).

CHRONOMÉTRER UN MORCEAU DE CODE

```
unsigned long t0 = micros();
analogRead(A0); // code à mesurer
unsigned long duree = micros() - t0;
```

CLIGNOTER SANS BLOQUER

```
const int LED = 13;
const unsigned long PERIODE = 500;
unsigned long precedent = 0; ❶
bool etat = LOW;
```

```
void setup() {
  pinMode(LED, OUTPUT);
}
```

```
void loop() {
  unsigned long t = millis();
  if (t - precedent >= PERIODE) { ❷
    precedent = t;
    etat = !etat;
    digitalWrite(LED, etat);
  }
  ❸ Lire un bouton, un capteur...
}
```

- ❶ un instant se range **toujours** dans un **unsigned long** ;
- ❷ la **soustraction** reste juste quand **millis()** repasse par 0 : avec **precedent** = 4 294 967 000 et **t** = 204, la différence vaut 500. Ne pas écrire **t >= precedent + PERIODE** ❶ ;
- ❸ **loop()** n'attend jamais : elle tourne des milliers de fois par seconde et peut tout surveiller.

PLUSIEURS TÂCHES À DES RYTHMES DIFFÉRENTS

```
unsigned long tLed = 0, tMesure = 0;

void loop() {
  unsigned long t = millis();
  if (t - tLed >= 250) { // 250 ms
    tLed = t;
    digitalWrite(13, !digitalRead(13));
  }
  if (t - tMesure >= 2000) { // 2 s
    tMesure = t;
    Serial.println(analogRead(A0));
  }
}
```

ANTI-REBOND SANS BLOQUER

```
const int BOUTON = 2; // vers GND
const unsigned long STABLE = 30; // ms
int etatStable = HIGH, dernierLu = HIGH;
unsigned long tChange = 0;
```

```
bool appui() { // 1 fois par appui
  int lu = digitalRead(BOUTON);
  if (lu != dernierLu) { ❶
    dernierLu = lu;
    tChange = millis();
  }
  if (millis() - tChange >= STABLE
      && lu != etatStable) { ❷
    etatStable = lu;
    return etatStable == LOW;
  }
  return false;
}
```

```
void setup() {
  pinMode(BOUTON, INPUT_PULLUP);
  Serial.begin(9600);
}
```

```
void loop() {
  if (appui()) Serial.println("Appui");
}
```

- ❶ la lecture change : on relance le chronomètre ;
- ❷ elle n'a plus bougé depuis 30 ms : c'est le nouvel état.

Clé en main : bibliothèque **Bounce2**.

MACHINE À ÉTATS : UN FEU TRICOLEURE

```
enum Etat { VERT, ORANGE, ROUGE };
Etat etat = VERT;
unsigned long debut = 0;
```

```
void allumer(int b) { // 9, 10 ou 11
  for (int p = 9; p <= 11; p++)
    digitalWrite(p, p == b);
}
```

```
void changer(Etat nouveau) {
  etat = nouveau;
  debut = millis();
}
```

```
void setup() {
  for (int p = 9; p <= 11; p++)
    pinMode(p, OUTPUT);
}
```

```
void loop() {
  unsigned long dt = millis() - debut;
  switch (etat) {
    case VERT:
      allumer(9); ❶
      if (dt >= 5000) changer(ORANGE);
      break;
    case ORANGE:
      allumer(10);
      if (dt >= 1000) changer(ROUGE);
      break;
    case ROUGE:
      allumer(11);
      if (dt >= 4000) changer(VERT);
      break;
  }
}
```

- ❶ À chaque tour : agir selon l'état courant, puis tester les **transitions** (les **if**). Ajouter un bouton pignon = ajouter une transition, sans rien bloquer.

LES TROIS MÉMOIRES (UNO R3)

Flash le programme (0,5 Ko pour le chargeur), les constantes **PROGMEM** ; conservée hors tension

SRAM variables, pile des appels, tas (**String**) ; effacée à l'arrêt

EEPROM réglages à conserver ; 100 000 écritures par case, 3,3 ms chacune

Après chaque compilation, l'IDE affiche :

```
Le croquis utilise n octets (p%) de
l'espace de stockage de programmes.
Le maximum est de 32256 octets.
Les variables globales utilisent n octets
(p%) de mémoire dynamique, ce qui laisse
n octets pour les variables locales.
Le maximum est de 2048 octets.
```

Au-delà de 75 % de SRAM : « La mémoire disponible faible, des problèmes de stabilité pourraient survenir. » (sic). La pile a besoin de la place restante : plantages aléatoires ❶

ÉCONOMISER LA SRAM

```
Serial.println(F("Texte en flash"));
```

```
const byte t[] PROGMEM = {1, 2, 4, 8};
byte v = pgm_read_byte(&t[2]); → 4
```

- **F()** pour tout texte fixe passé à **print()** ;
- le plus petit type suffisant (**byte** plutôt qu'**int**) ;
- éviter **String** et les grands tableaux ; un écran OLED 128 × 64 réserve 1 Ko (la moitié de la SRAM !).

EEPROM : GARDER UNE VALEUR APRÈS EXTINCTION

```
#include <EEPROM.h>
```

```
void setup() {
  EEPROM.update(0, 42); // si différent
  byte v = EEPROM.read(0); → 42
  float c = 21.5;
  EEPROM.put(10, c); // cases 10..13
  EEPROM.get(10, c); // relecture
  int n = EEPROM.length(); → 1024
}
```

- **update()** n'écrit que si la valeur change ; **put()**, **get()** : une variable entière (ici un **float**, 4 octets) ; case jamais écrite : 255.
- 100 000 écritures par case : jamais à chaque tour de **loop()** ❶
- [R4] EEPROM de 8 Ko, même interface.

CHIEN DE GARDE (AVR)

```
#include <avr/wdt.h>
```

```
void setup() {
  wdt_enable(WDTO_2S); ❶
}
```

```
void loop() {
  wdt_reset(); ❷
  // ... travail normal
}
```

- ❶ si le programme se fige plus de 2 s, la carte **redémarre** d'elle-même ;
- ❷ « nourrir » le chien de garde à chaque tour de **loop()**.

Propre aux microcontrôleurs AVR (Uno R3, Nano, Mega). Des redémarrages en boucle sont signalés avec l'ancien chargeur d'amorçage de certains Nano : tester avant de s'y fier.

6 · Communiquer : série, I²C, SPI

MÉMENTO — ARDUINO

LIAISON SÉRIE : ENVOYER

Serial.begin(9600)

ouvrir à 9 600 bauds (bits/s) ; 115200 : 12 fois plus rapide

Serial.print(x) écrire en texte, sans retour à la ligne

Serial.println(x) puis passer à la ligne (\r\n)

Serial.print(78, HEX) → 4E ; BIN → 1001110 ; DEC par défaut

Serial.print(1.23456) → 1.23 : 2 décimales par défaut

Serial.print(1.23456, 4) → 1.2346

Serial.write(65) envoyer l'octet brut 65 (le caractère A)

Serial.print(F("...")) texte fixe laissé en flash

Serial.flush() attendre la fin de l'envoi

- Uno : broches 0 et 1 partagées avec l'USB ; tampon de réception de 64 octets.
- Cartes à USB natif (Leonardo, Micro...) : **while (!Serial) {}** dans **setup()** pour ne rien perdre.

LIAISON SÉRIE : RECEVOIR

```
void setup() {
  Serial.begin(9600);
  pinMode(13, OUTPUT);
}
```

```
void loop() {
  if (Serial.available() > 0) {
    String c =
      Serial.readStringUntil('\n');
    c.trim();
    if (c == "on")
      digitalWrite(13, HIGH);
    else if (c == "off")
      digitalWrite(13, LOW);
    else
      Serial.println("Inconnu : " + c);
  }
}
```

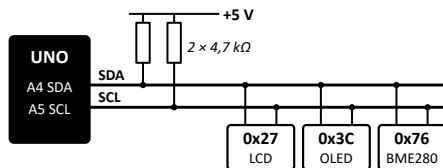
- 1 nombre d'octets reçus en attente ; **Serial.read()** en lit un (-1 s'il n'y a rien) ;
- 2 lit jusqu'au caractère donné, ou jusqu'au délai d'attente (1 s par défaut, **Serial.setTimeout()**) ; **Serial.parseInt()** lit un nombre ;
- 3 retire espaces et \r (fin de ligne « Les deux, NL et CR »).

MONITEUR ET TRACEUR SÉRIE

- Débit du moniteur = débit de **Serial.begin()**, sinon caractères illisibles.
- Fin de ligne ajoutée à l'envoi : *Pas de fin de ligne* · Nouvelle ligne (\n) · Retour chariot (\r) · Les deux, NL et CR.
- **Traceur série** (Outils › Traceur série) : une ligne par mesure, valeurs séparées par une virgule, une espace ou une tabulation, étiquetées **nom :**

```
Serial.print("temp:");
Serial.print(t);
Serial.print(",hum:");
Serial.println(h);
→ temp:21.50,hum:40.00
```

BUS I²C (BIBLIOTHÈQUE WIRE)



- Deux fils + la masse : **SDA** (données, A4) et **SCL** (horloge, A5) sur Uno ; tous les modules en parallèle.
- Chaque module a une **adresse** sur 7 bits, souvent réglable ; 0x08 à 0x77 utilisables.
- Résistances de tirage vers VCC (souvent 4,7 kΩ), en général déjà présentes sur les modules.
- 100 kHz par défaut ; **Wire.setClock(400000)** : 400 kHz. Fils courts (bus prévu pour l'intérieur d'un appareil).

```
#include <Wire.h>
```

```
void setup() { // scanner I2C
  Serial.begin(9600);
  Wire.begin(); // pilote du bus
  for (byte a = 8; a < 120; a++) {
    Wire.beginTransmission(a);
    if (Wire.endTransmission() == 0) {
      Serial.print("Module en 0x");
      Serial.println(a, HEX);
    }
  }
}
```

```
void loop() {}
```

- 1 appel à l'adresse a ; **endTransmission()** renvoie 0 succès · 1 données trop longues (tampon de 32 octets) · 2 adresse sans réponse · 3 donnée refusée · 4 autre erreur · 5 délai dépassé.

LIRE DES REGISTRES

```
Wire.beginTransmission(0x68); // module
Wire.write(0x00); // registre
Wire.endTransmission(false); // garder
Wire.requestFrom(0x68, 3); // 3 octets
while (Wire.available()) {
  byte b = Wire.read();
}
```

false : redémarrage sans libérer le bus, entre l'écriture du n° de registre et la lecture.

Adresses courantes, à vérifier sur le module : LCD à adaptateur PCF8574 0x27 ou 0x3F ; OLED SSD1306 0x3C ; BME280 0x76 ou 0x77 ; horloge DS3231 et MPU-6050 0x68.

UN SECOND PORT SÉRIE

```
#include <SoftwareSerial.h>
SoftwareSerial gps(4, 3); // RX, TX

void setup() {
  Serial.begin(9600); // vers Le PC
  gps.begin(9600); // vers Le module
}

void loop() { // relais
  if (gps.available())
    Serial.write(gps.read());
}
```

Un seul port logiciel écoute à la fois (**listen()**) ; peu fiable à haut débit. Mega : ports matériels **Serial1** à **Serial3**.

BUS SPI (BIBLIOTHÈQUE SPI)

- 4 fils : **SCK** horloge (13), **COPI** ou **MOSI** (11), **CIP0** ou **MISO** (12), **CS** sélection : une broche par module (10 par défaut).
- Rapide (plusieurs MHz), dans les deux sens à la fois : cartes SD, écrans TFT, lecteurs RFID, modules radio.
- Sur AVR, la broche 10 doit rester une **sortie** : en entrée à l'état bas, le SPI bascule en esclave

```
#include <SPI.h>
const int CS = 10;

void setup() {
  pinMode(CS, OUTPUT);
  digitalWrite(CS, HIGH); // Libre
  SPI.begin();
}

byte lire(byte reg) {
  SPISettings r(1000000, MSBFIRST,
    SPI_MODE0);
  SPI.beginTransaction(r);
  digitalWrite(CS, LOW); // sélection
  SPI.transfer(reg);
  byte v = SPI.transfer(0);
  digitalWrite(CS, HIGH);
  SPI.endTransaction();
  return v;
}

void loop() {}
```

- 1 vitesse (1 MHz), ordre des bits, mode (0 à 3) : lire la fiche du composant ;
- 2 on envoie un octet quelconque pour en recevoir un.

QUEL BUS CHOISIR ?

UART TX et RX croisés ; 1 face à 1 ; 9 600 à 115 200 bauds ; GPS, Bluetooth HC-05, ordinateur

I²C SDA, SCL ; plusieurs modules (adresses) ; 100 à 400 kHz ; écrans, capteurs, horloge

SPI 3 fils + 1 CS par module ; plusieurs MHz ; carte SD, écran TFT, RFID

1-Wire 1 fil + tirage 4,7 kΩ ; plusieurs (n° unique) ; ≈ 15 kbit/s ; sonde DS18B20

UART : le TX de l'un va au RX de l'autre ; masse commune.

NIVEAUX 5 V ↔ 3,3 V

- 5 V : Uno R3 et R4, Nano, Mega · 3,3 V : Nano ESP32, Nano 33, ESP32, la plupart des capteurs récents.
- Sortie 3,3 V → entrée 5 V : en général lue **HIGH** (seuil ≈ 3 V), marge faible.
- Sortie 5 V → entrée 3,3 V : **jamais en direct** ! diviseur (1 kΩ / 2 kΩ, p. 2) ou adaptateur de niveaux.
- Beaucoup de modules ont régulateur et adaptateur intégrés : lire la tension indiquée sur VCC.

7 · Capteurs et actionneurs

MÉMENTO — ARDUINO

CAPTEURS COURANTS

CAPTEUR	LIAISON · OUTIL
Potentiomètre, joystick	analogique · <code>analogRead()</code>
Photorésistance (LDR)	analogique, diviseur 10 kΩ · <code>analogRead()</code>
LM35, TMP36 (température)	analogique · calcul ci-dessous
DHT11, DHT22 (temp., humidité)	1 fil propre au capteur · <i>DHT sensor library</i>
DS18B20 (température, étanche)	1-Wire + 4,7 kΩ · <i>OneWire</i> , <i>DallasTemperature</i>
BME280 (pression, temp., hum.)	I ² C ou SPI · <i>Adafruit BME280 Library</i>
HC-SR04 (distance)	2 broches, Trig et Echo · <code>pulseIn()</code>
HC-SR501 (présence, PIR)	numérique · <code>digitalRead()</code>
Codeur rotatif	2 broches + bouton · interruptions
DS3231 (horloge)	I ² C · <i>RTClib</i>
RC522 (badges RFID)	SPI, 3,3 V ❶ · <i>MFRC522</i>

En italique : nom exact dans le gestionnaire de bibliothèques (p. 8).

TEMPÉRATURE ANALOGIQUE

```
int v = analogRead(A0);
float mv = v * 5000.0 / 1024; // mV
float tLM35 = mv / 10; // °C
float tTMP36 = (mv - 500) / 10; // °C
```

LM35 : 10 mV/°C, 0 mV à 0 °C. TMP36 : 10 mV/°C, 500 mV à 0 °C (750 mV → 25 °C). Plus de finesse : `analogReference(INTERNAL)` (1,1 V) et 1100 au lieu de 5000.

TEMPÉRATURE ET HUMIDITÉ : DHT22

```
#include <DHT.h>
DHT dht(2, DHT22); // broche, modèle
```

```
void setup() {
  Serial.begin(9600);
  dht.begin();
}
```

```
void loop() {
  delay(2000);
  float h = dht.readHumidity(); // %
  float t = dht.readTemperature(); // °C
  if (isnan(h) || isnan(t)) {
    Serial.println("Erreur");
    return;
  }
  Serial.print(t);
  Serial.print(" °C ");
  Serial.print(h);
  Serial.println(" %");
}
```

❶ le DHT22 ne fournit pas plus d'une mesure toutes les 2 s ;

❷ `isnan()` : « n'est pas un nombre », lecture ratée (câblage, broche).

DISTANCE : HC-SR04

```
const int TRIG = 9, ECHO = 10;
```

```
void setup() {
  Serial.begin(9600);
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
}
```

```
void loop() {
  digitalWrite(TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  unsigned long d =
    pulseIn(ECHO, HIGH, 30000); ❷
  if (d > 0) {
    float cm = d * 0.0343 / 2; ❸
    Serial.println(cm);
  }
  delay(100);
}
```

❶ impulsion de 10 µs : le module émet ses ultrasons ;

❷ durée de l'écho en µs (aller-retour) ; 0 si rien en 30 ms ;

❸ son : 343 m/s à 20 °C, soit 0,0343 cm/µs ; ÷ 2 pour l'aller seul ; soit cm ≈ durée / 58.

ACTIONNEURS COURANTS

LED, LED RVB `digitalWrite`, `analogWrite` + résistance

Buzzer actif : `digitalWrite` · passif : `tone()`

Servomoteur (SG90...) *Servo* ; ci-contre

Moteur à courant continu pont en H : L298N, L293D, TB6612FNG

Pas à pas 28BYJ-48 module ULN2003 ; *Stepper*, *AccelStepper*

Pas à pas NEMA 17 pilote A4988 ou DRV8825 (STEP, DIR) ; *AccelStepper*

Relais module avec transistor et diode ; secteur : danger ❶

Ruban WS2812B 1 fil de données ; *Adafruit NeoPixel*, *FastLED*

Écran LCD 16 × 2 *LiquidCrystal* (6 fils) ; adaptateur I²C : 4 fils

Écran OLED 128 × 64 I²C ; *Adafruit SSD1306* + *Adafruit GFX Library*

SERVOMOTEUR

```
#include <Servo.h>
Servo bras;
```

```
void setup() {
  bras.attach(9); ❶
}
```

```
void loop() {
  int v = analogRead(A0);
  int angle = map(v, 0, 1023, 0, 180);
  bras.write(angle); ❷
  delay(15);
}
```

❶ fil de commande sur une broche quelconque ; jusqu'à 12 servos ; sur Uno, la bibliothèque **supprime la MLI de 9 et 10** ;

❷ angle de 0 à 180° ; `bras.writeMicroseconds(1500)` : durée d'impulsion (544 à 2 400 µs par défaut pour 0 à 180°).

Fils : rouge + 5 V, noir ou marron GND, jaune, orange ou blanc : commande. Un servo tire de fortes pointes de courant : au-delà d'un ou deux, alimentation séparée, masses reliées.

MOTEUR CC : PONT EN H

IN1 ≠ IN2 le moteur tourne ; on inverse les deux pour changer de sens

IN1 = IN2 arrêt rapide (freinage)

EN à LOW roue libre

EN en MLI vitesse : `analogWrite(EN, 0...255)`

```
const int EN = 5; // broche MLI
const int IN1 = 7, IN2 = 8;
```

```
void moteur(int v) { // -255 à 255
  digitalWrite(IN1, v > 0);
  digitalWrite(IN2, v < 0);
  analogWrite(EN, min(abs(v), 255));
}
```

```
void setup() {
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  moteur(-150); // recule
}
```

```
void loop() {}
```

Moteur alimenté à part, masses reliées. Module L298N : retirer le cavalier ENA pour régler la vitesse.

ÉCRAN LCD 16 × 2

```
#include <LiquidCrystal.h>
// RS, E, D4, D5, D6, D7
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
```

```
void setup() {
  lcd.begin(16, 2); // taille
  lcd.print("Bonjour !");
}
```

```
void loop() {
  lcd.setCursor(0, 1); // 2e ligne
  lcd.print(millis() / 1000);
  lcd.print(" s");
}
```

Contraste : potentiomètre de 10 kΩ sur V0 (écran vide ou rectangles pleins sinon). Pas d'accents dans le jeu de caractères d'origine. Version I²C (PCF8574) : bibliothèque « *LiquidCrystal I2C* », dont il existe plusieurs variantes : partir de l'exemple fourni.

RUBAN DE LED WS2812B

```
#include <Adafruit_NeoPixel.h>
// 30 LED, données sur la broche 6
Adafruit_NeoPixel ruban(30, 6,
  NEO_GRB + NEO_KHZ800);
```

```
void setup() {
  ruban.begin();
  ruban.setBrightness(50); // 0...255
}
```

```
void loop() { // un point qui court
  for (int i = 0; i < 30; i++) {
    ruban.clear();
    ruban.setPixelColor(i,
      ruban.Color(255, 0, 0)); // rouge ❶
    ruban.show();
    delay(50);
  }
}
```

❶ rien ne change avant `show()`. Jusqu'à 60 mA par LED en blanc à pleine luminosité (1,8 A pour 30) : alimentation 5 V dédiée, masses reliées, condensateur de 500 à 1 000 µF sur l'alimentation, 300 à 500 Ω sur le fil de données.

8 · Cartes, bibliothèques, dépannage

CHOISIR SA CARTE					
CARTE	PUCE	E/S	NUM. (MLI) · ANA.	FLASH / RAM	PARTICULARITÉS
Uno R3	ATmega328P · 16 MHz	5 V	14 (6) · 6	32 Ko / 2 Ko	la référence ; USB-B
Uno R4 Minima	RA4M1 Arm · 48 MHz	5 V	14 (6) · 6	256 Ko / 32 Ko	CNA 12 bits, CAN jusqu'à 14 bits, bus CAN ; USB-C ; 8 mA/broche
Uno R4 WiFi	RA4M1 + ESP32-S3	5 V	14 (6) · 6	256 Ko / 32 Ko	+ Wi-Fi, Bluetooth, matrice de 12 × 8 LED
Nano	ATmega328 · 16 MHz	5 V	14 (6) · 8	32 Ko / 2 Ko	18 × 45 mm, mini-USB
Nano Every	ATmega4809 · 20 MHz	5 V	22 (5) · 8	48 Ko / 6 Ko	format Nano, micro-USB
Nano ESP32	ESP32-S3 · 240 MHz	3,3 V	14 · 8	16 Mo / 512 Ko	Wi-Fi, Bluetooth ; MLI sur toutes les broches (5 à la fois)
Mega 2560	ATmega2560 · 16 MHz	5 V	54 (15) · 16	256 Ko / 8 Ko	4 ports série matériels
Leonardo	ATmega32U4 · 16 MHz	5 V	20 (7) · 12	32 Ko / 2,5 Ko	USB natif : clavier, souris

Caractéristiques : documentation officielle (docs.arduino.cc).

BIBLIOTHÈQUES	
- Installer : Outils › Gérer les bibliothèques... (Ctrl+Shift+I), chercher, Installer ; accepter <i>Tout installer</i> pour les dépendances. - Depuis une archive : Croquis › Importer une bibliothèque › Ajouter la bibliothèque .ZIP... - Utiliser : #include <Nom.h> en tête du croquis ; exemples dans Fichier › Exemples. - Fournies avec les cartes AVR : EEPROM, SoftwareSerial, SPI, Wire. Officielles : Servo, LiquidCrystal, Stepper.	
DHT sensor library	DHT11, DHT22 (demande Adafruit Unified Sensor)
OneWire + DallasTemperature	sondes DS18B20
Adafruit NeoPixel · FastLED	rubans WS2812B
Adafruit SSD1306	écrans OLED (demande Adafruit GFX Library)
RTCLib · AccelStepper	horloges DS3231 · moteurs pas à pas
Bounce2 · MFRC522	anti-rebond · lecteur RFID RC522

AJOUTER UNE FAMILLE DE CARTES	
- Outils › Carte › Gestionnaire de cartes : « Arduino AVR Boards » (Uno R3, Nano, Mega...), « Arduino UNO R4 Boards », « Arduino ESP32 Boards » (Nano ESP32). - Autres ESP32 : Fichier › Préférences › URL de gestionnaire de cartes supplémentaires : https://espressif.github.io/arduino-esp32/package_esp32_index.json puis installer esp32.	

MÉTHODE
- Avancer par petites étapes : un composant, un test, puis le suivant. - Partir d'un exemple qui fonctionne, puis le modifier. - Déboguer avec Serial.println() : afficher valeurs et étapes franchies. - Noms explicites, constantes pour les broches, câblage décrit en commentaire en tête du croquis ; (Ctrl+T) pour mettre en forme. - Garder les versions qui marchent (copie datée, Git). - Tester sans matériel : simulateurs en ligne (Wokwi, Tinkercad Circuits). - Référence : docs.arduino.cc (langage, cartes, tutoriels) ; entraide : forum.arduino.cc (section francophone).

MESSAGES D'ERREUR FRÉQUENTS	
COMPILATION (ERREUR DE COMPILATION)	
'Led' was not declared in this scope	nom mal écrit, casse (Led ≠ led), variable déclarée dans un autre bloc
expected ';' before '}' token	point-virgule oublié en fin de ligne
expected '}' at end of input	accolade fermante manquante
a function-definition is not allowed here before '{' token	fonction précédente non refermée
'Servo' does not name a type	#include oublié
Servomoteur.h: No such file or directory	bibliothèque non installée ou nom mal écrit
redefinition of 'void setup()'	deux onglets définissent setup()
stray '\302' in program	caractère invisible collé depuis une page web : retaper la ligne
warning: integer overflow in expression	calcul trop grand pour un int (p. 4)
TÉLÉVERSEMENT (FAILED UPLOADING)	
programmer is not responding · not in sync	mauvais port ou mauvaise carte ; quelque chose sur la broche 0 ; certains Nano : Outils › Processor › ATmega328P (Old Bootloader)
cannot open port COM3	port occupé par un autre logiciel, carte débranchée, mauvais port
Toujours lire la première erreur : les suivantes en découlent souvent.	

DÉPANNAGE : SYMPTÔME → PISTE	
Aucun port dans Outils › Port	câble de charge seule ; autre câble, autre prise USB ; pilote CH340 (clone)
Caractères illisibles	débit du moniteur ≠ Serial.begin()
Rien ne s'affiche	Serial.begin() oublié ; mauvais port ; USB natif : while (!Serial)
La LED reste éteinte	LED à l'envers ; pinMode oublié ; mauvaise broche
Lecture qui change seule	entrée flottante : INPUT_PULLUP ou résistance de tirage
Un appui compte plusieurs fois	rebonds : anti-rebond (p. 5)
Redémarre quand le moteur part	chute de tension : alimentation séparée, masse commune
Plante après des heures	SRAM épuisée (String), int qui déborde, millis() mal comparé
Module I²C muet	adresse (scanner p. 6), SDA et SCL inversés, alimentation 3,3 / 5 V

EN LIGNE DE COMMANDE : ARDUINO-CLI	
<pre>arduino-cli core update-index arduino-cli core install arduino:avr arduino-cli board list # cartes et ports arduino-cli compile --fqbn arduino:avr:uno Blink arduino-cli upload -p COM3 --fqbn arduino:avr:uno Blink arduino-cli lib install "DHT sensor library" arduino-cli monitor -p COM3 --config 115200</pre>	

PROGRESSER : DIX MONTAGES	
1	Blink : la LED intégrée clignote (p. 1).
2	Feu tricolore : trois LED et leurs résistances, avec delay() , puis avec millis() (p. 5).
3	Bouton qui allume une LED, puis bascule avec anti-rebond (p. 3, 5).
4	Variateur : un potentiomètre règle la luminosité par MLI (p. 3).
5	Veilleuse automatique à photorésistance (p. 3).
6	Thermomètre : valeurs au moniteur, puis au traceur série (p. 6, 7).
7	Radar de recul : HC-SR04 et buzzer dont le rythme suit la distance (p. 7).
8	Aiguille : un servomoteur suit un potentiomètre (p. 7).
9	Station météo : DHT22 et écran LCD (p. 7).
10	Petit robot : deux moteurs, pont en H, capteur de distance, machine à états (p. 5, 7).